



MACHINE LEARNING

MKT3434

Homework Assignment #3

Betül DAŞDEMİR

20067502

Table of Contents

Introduction	2
Methods Used and System Architecture	2
Layer Architectures	2
Advanced Deep Learning Features	3
Typical CNN Architecture and Justification of Design Choices	4
Advantages of the Architectural Choices	5
Results and Screenshots	5

Introduction

In this study, deep learning architectures including MLP, CNN, and RNN structures were dynamically designed, trained, and evaluated through a PyQt6-based GUI. Features such as layer-based configuration, optimizer and scheduler selection, regularization techniques, model saving/loading, and training visualization were integrated.

Methods Used and System Architecture

- Dynamic layer addition and removal (Dense, Conv2D, LSTM, GRU, Dropout, Flatten, MaxPooling)
- Optimizer selection: Adam, SGD, RMSprop
- Learning rate scheduling: Step decay, exponential decay
- Regularization: L2 and Dropout
- Early stopping support
- Model saving/loading: .h5 (weights), .json (architecture)
- Data augmentation: rotation, flipping, zoom
- Transfer learning: usage of VGG16 and ResNet
- GAN: Generator and Discriminator architectures integrated via the GUI

Layer Architectures

✓MLP:

- Dense(128, activation='relu')
- Dropout(0.5)
- Dense(64, activation='tanh')
- Dense(10, activation='softmax')

✓CNN:

- Conv2D(32, (3,3), activation='relu')
- MaxPooling2D((2,2))
- Conv2D(64, (3,3), activation='relu')
- MaxPooling2D((2,2))
- Flatten()
- Dense(128, activation='relu')
- Dropout(0.5)
- Dense(10, activation='softmax')

✓RNN (LSTM/GRU):

- LSTM(64, return_sequences=True)
- Dropout(0.5)
- GRU(32)
- Dense(1, activation='sigmoid')

Advanced Deep Learning Features

a) Model Configuration and Layer Management

The updated deep learning module is no longer limited to basic MLP (Multilayer Perceptron) structures; it also enables the creation of advanced neural network architectures for both visual and temporal datasets. Through an intuitive GUI (Graphical User Interface), users can build multi-layered networks, monitor training processes, and export models for integration into other projects.

Layer Addition Support: Users can flexibly design artificial neural network models by specifying the number of neurons and activation functions (ReLU, Sigmoid, Tanh) for each layer via the GUI.

Visual Data Processing: Conv2D (convolutional) and MaxPooling2D (pooling) layers are supported for image-based data. Parameters such as the number of filters and kernel size can be customized.

Temporal Data Support: LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) layers are supported for time series data, offering significant advantages in analyzing signals, text, and time-based events.

Model Saving/Loading: Created models can be saved in .h5 (weights) and .json (architecture) formats and reloaded into the GUI for continued training or deployment.

Dynamic Layer Editing: Users can update the architecture by modifying existing layers, removing them, or adding new ones.

b) Training Process and Optimization

Optimization Algorithms: During training, users can select from various optimization algorithms such as Adam, SGD, and RMSprop.

Learning Rate Scheduling: Learning rate adjustment is supported through Step Decay and Exponential Decay strategies, ensuring a dynamic learning process.

Regularization: Dropout and L2 regularization techniques are supported to prevent overfitting.

Validation Monitoring: Loss values based on the validation dataset are tracked during training, enabling early detection of overfitting.

c) Training Visualization and Evaluation

Training Graphs: Epoch-based loss and accuracy metrics are visualized in real time using the matplotlib library.

Test Results: Upon completion of training, performance metrics such as accuracy and scores are computed using the test dataset.

Confusion Matrix: For classification tasks, a confusion matrix is generated by comparing predicted and actual labels.

d) Transfer Learning and Data Augmentation

Transfer Learning: Pre-trained models (e.g., VGG16, ResNet) can be integrated into the GUI and fine-tuned on custom datasets.

Data Augmentation: Techniques such as rotation, flipping, and zooming are applied to increase diversity in visual datasets, thereby enhancing the model's generalization capability.

Typical CNN Architecture and Justification of Design Choices

A typical Convolutional Neural Network (CNN) architecture used for image processing is structured as follows, along with the rationale behind each component:

Conv2D Layers: These layers extract visual features. Layers with 32, 64, and 128 filters allow for the capture of features at varying levels. A (3x3) kernel size is effective for capturing local patterns and is computationally efficient.

MaxPooling2D Layers: (2x2) pooling reduces the dimensionality and helps prevent overfitting.

Dropout: Applied at a rate of 50%, dropout layers placed after dense layers increase the model's resilience against overfitting.

Flatten: Converts 2D feature maps into 1D vectors to serve as input for dense layers.

Dense (Fully Connected) Layers: These layers process features for classification, producing class probabilities through a softmax activation function at the output layer.

Advantages of the Architectural Choices

Feature Extraction: Multi-layered Conv2D and MaxPooling structures enable hierarchical learning of various structural characteristics in the data.

Generalization Ability: With regularization techniques like dropout, the model can generalize well even on small datasets.

Balance of Efficiency and Accuracy: The architecture offers a balanced performance in terms of training time, model complexity, and accuracy, making it ideal for image classification tasks.

Results and Screenshots

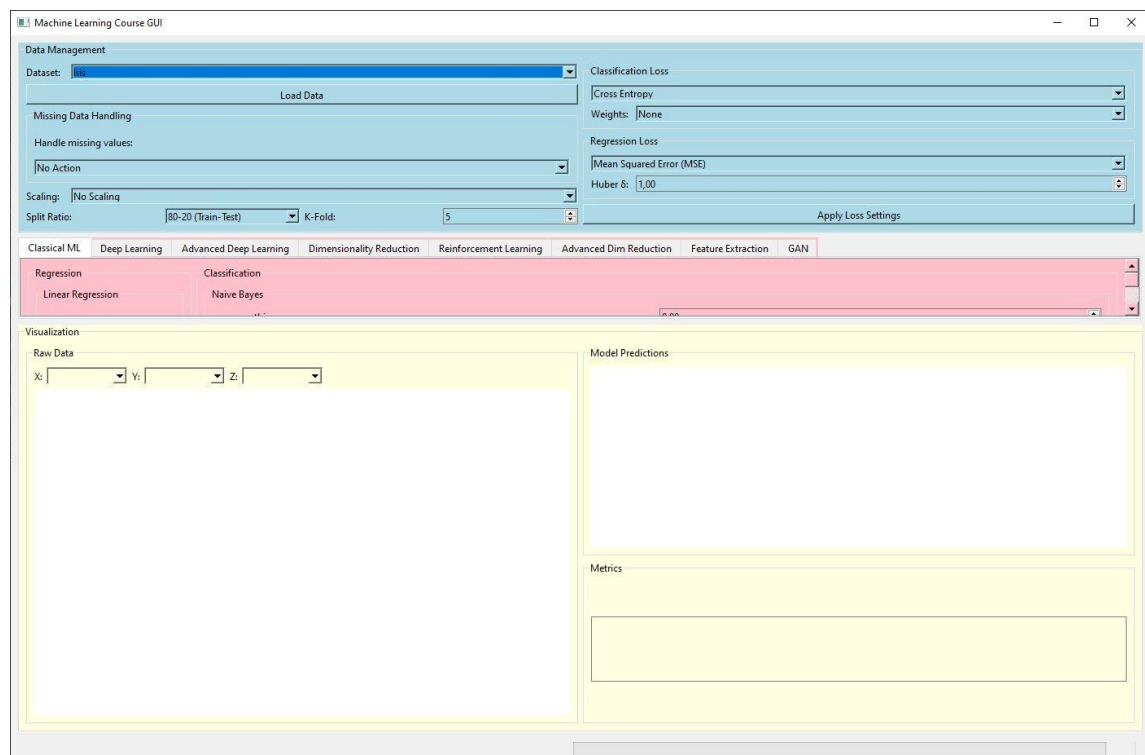


Figure 1-New GUI

The Deep Learning and Advanced Deep Learning tabs have been activated in this version. Users can modify key parameters and dynamically add neural network layers

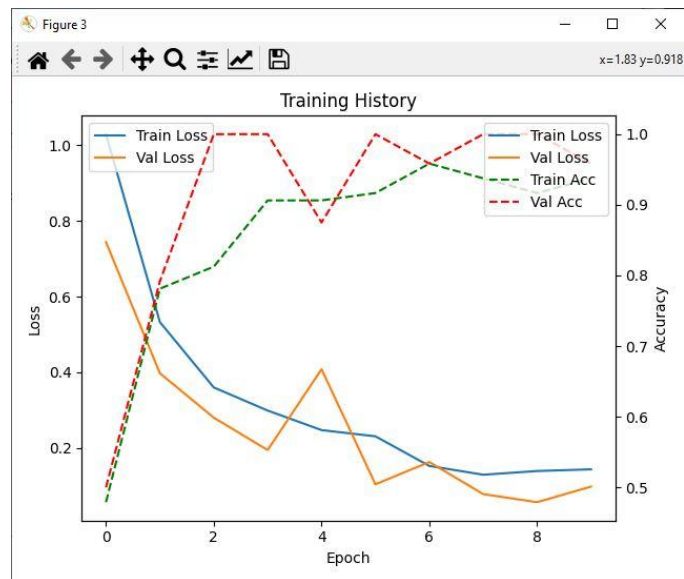


Figure 2- RNN Training Output

The loss and accuracy values of the trained model are displayed on a single plot. This allows simultaneous monitoring of both error and performance during training.

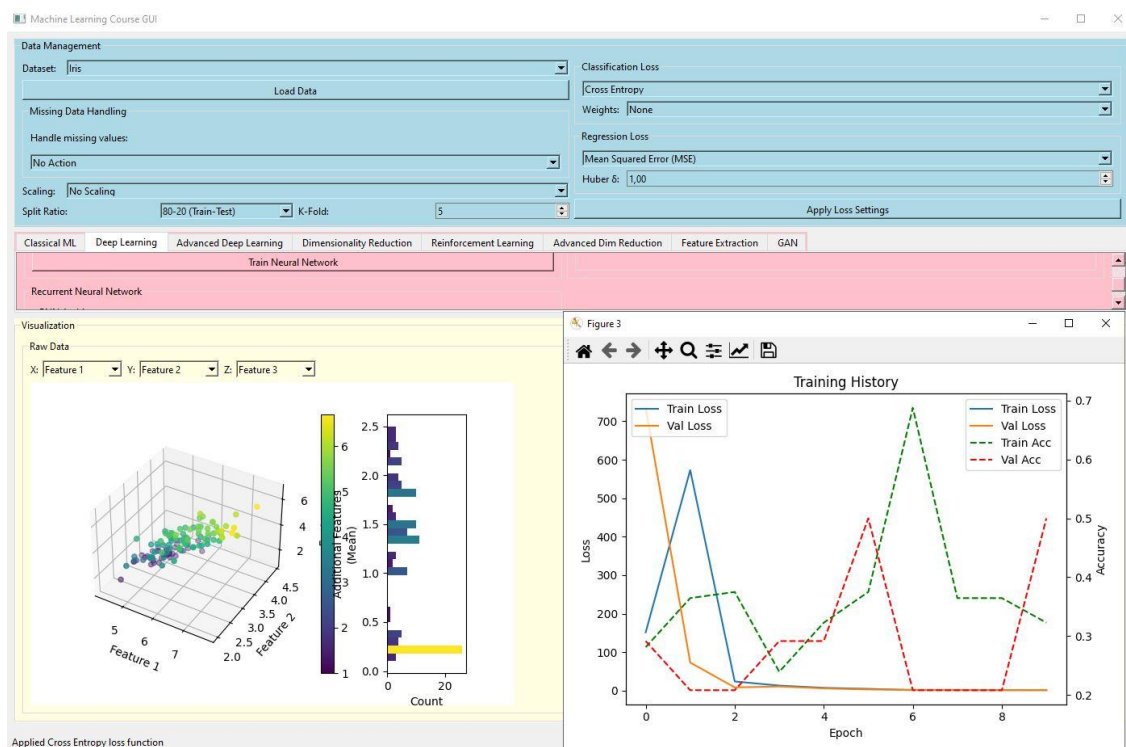


Figure 3-CNN

On the left, a 3D projection of the dataset is shown, while the right displays the training history of the RNN model. The fluctuating validation metrics may indicate potential overfitting during training.

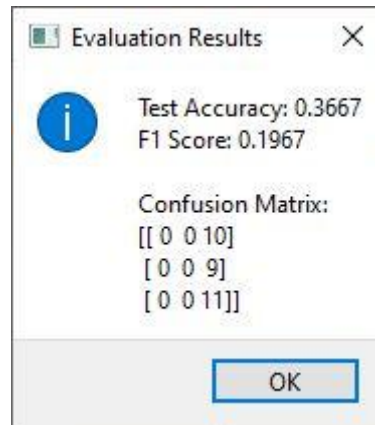


Figure 4-Evaluate Output

The model yielded low test accuracy and F1 score, indicating poor class separation. The confusion matrix shows that predictions are concentrated in a single class.

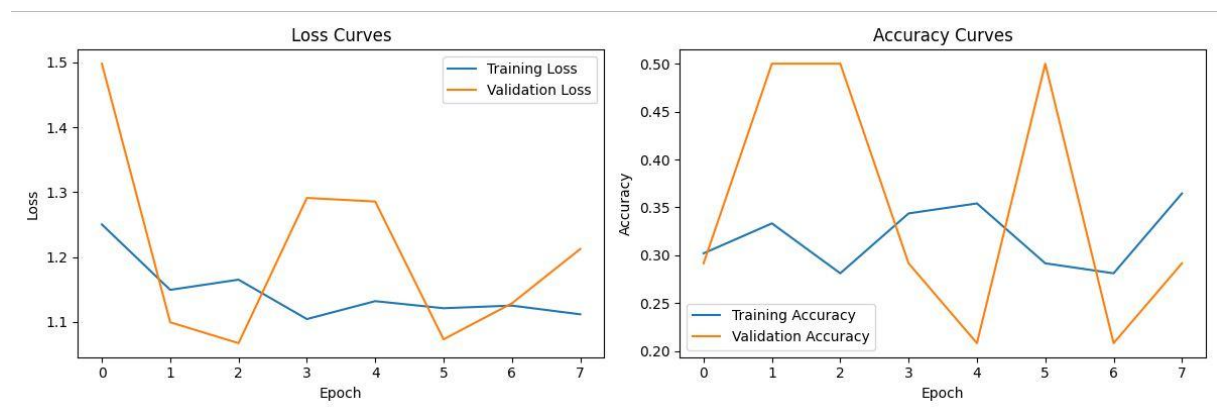


Figure 5-ADAM with RestNET50 Output

The model was trained using the Adam optimizer with fine-tuning applied. Fluctuations in the validation curves suggest the model still requires stabilization.

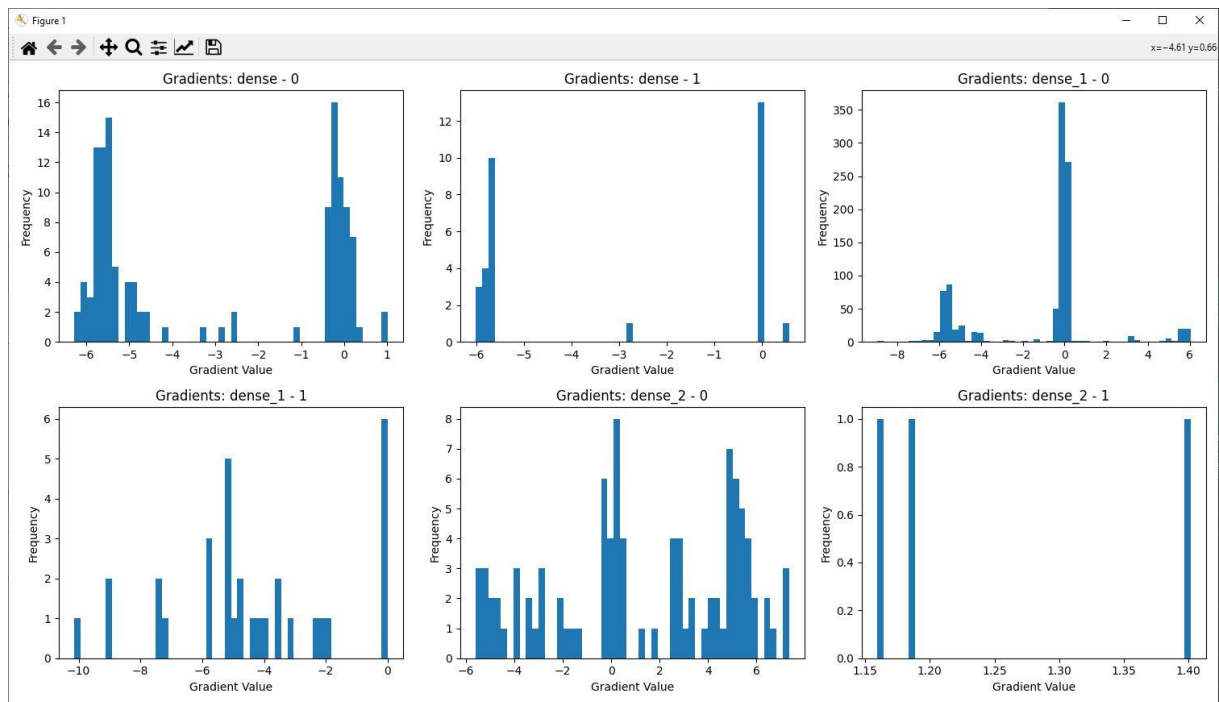


Figure 60-ADAM Out

This figure shows the gradient distributions of various dense layers during training. Most gradients are concentrated between -2 and 1, indicating stable weight updates across layers.

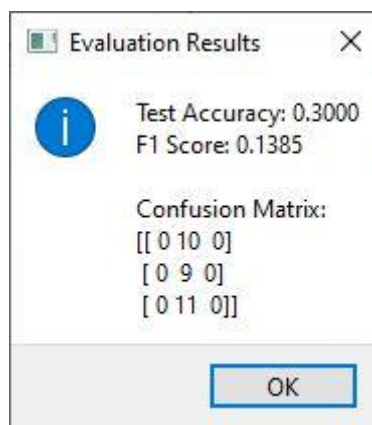


Figure 7- SGD Output

The model trained using SGD showed low accuracy and F1 score. The confusion matrix indicates poor class separation and frequent misclassifications.

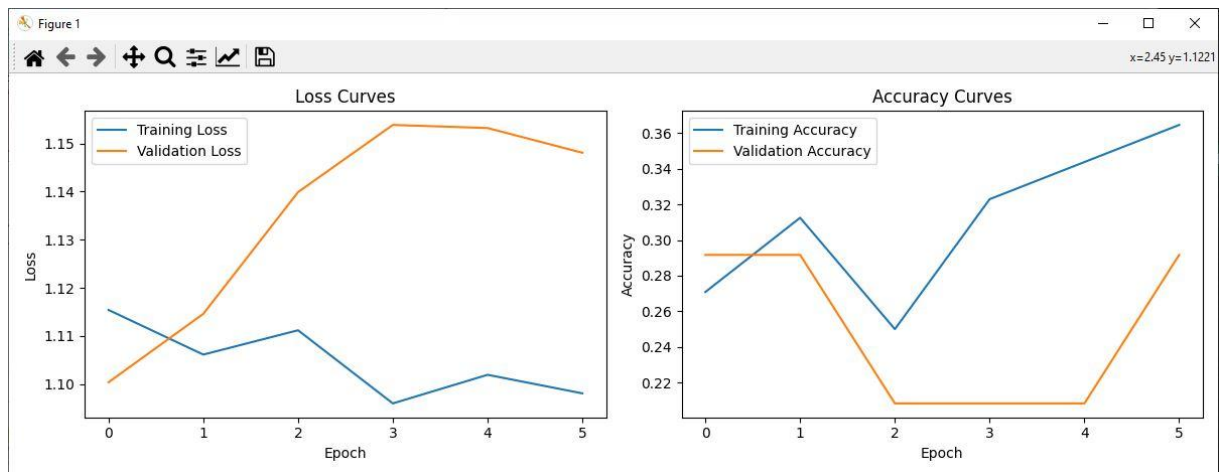


Figure 8-SGD with RestNET50 Output

Validation loss consistently increases while validation accuracy remains low. This indicates that the model fits the training data but struggles to generalize.

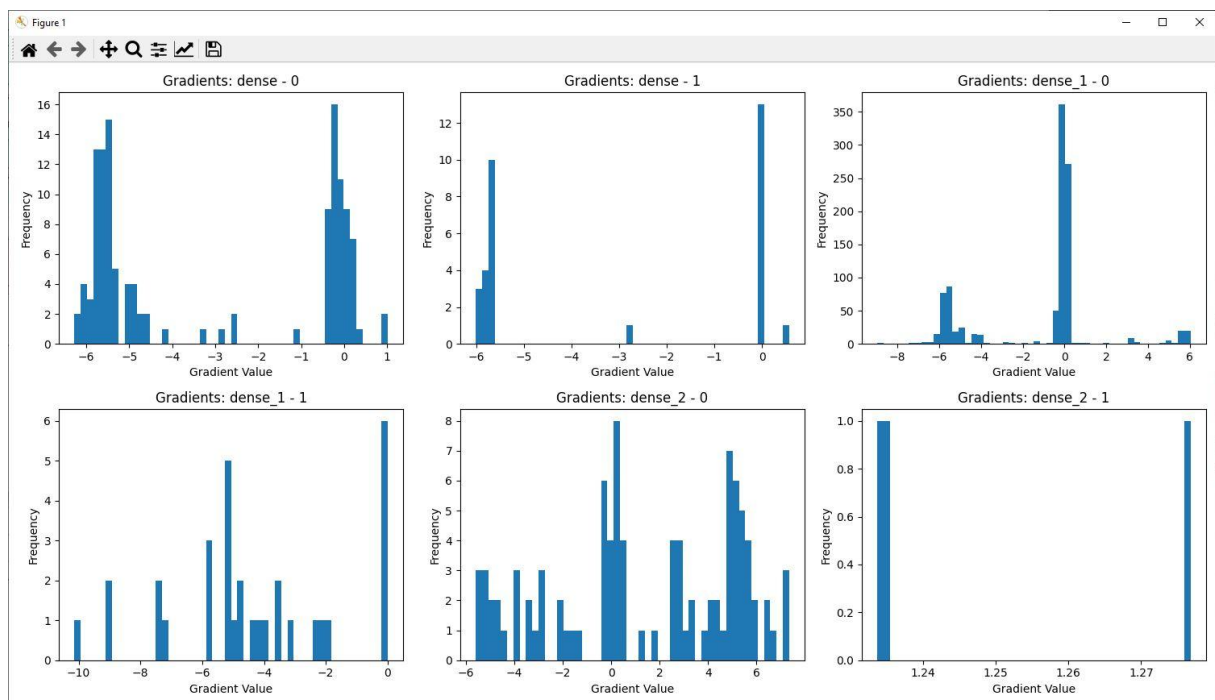


Figure 9-SGD Out

The gradient distributions across layers appear irregular and skewed under SGD optimization. This may indicate unstable updates and inconsistent learning behavior.

```

3/3 ██████████ 0s 13ms/step - accuracy: 0.3763 - loss: 1.1197 - val_accuracy: 0.2083 - val_loss: 1.2856
Epoch 6/10
3/3 ██████████ 0s 14ms/step - accuracy: 0.2826 - loss: 1.1276 - val_accuracy: 0.5000 - val_loss: 1.0734
Epoch 7/10
3/3 ██████████ 0s 12ms/step - accuracy: 0.2734 - loss: 1.1246 - val_accuracy: 0.2083 - val_loss: 1.1288
Epoch 8/10
3/3 ██████████ 0s 13ms/step - accuracy: 0.3893 - loss: 1.1074 - val_accuracy: 0.2917 - val_loss: 1.2125
1/1 ██████████ 0s 37ms/step
QCoreApplication::exec: The event loop is already running
QCoreApplication::exec: The event loop is already running
Epoch 1/10
3/3 ██████████ 0s 53ms/step - accuracy: 0.2526 - loss: 1.1208 - val_accuracy: 0.2917 - val_loss: 1.1003
Epoch 2/10
3/3 ██████████ 0s 10ms/step - accuracy: 0.3242 - loss: 1.1042 - val_accuracy: 0.2917 - val_loss: 1.1146
Epoch 3/10
3/3 ██████████ 0s 12ms/step - accuracy: 0.2539 - loss: 1.1076 - val_accuracy: 0.2083 - val_loss: 1.1399
Epoch 4/10
3/3 ██████████ 0s 11ms/step - accuracy: 0.3177 - loss: 1.0967 - val_accuracy: 0.2083 - val_loss: 1.1539
Epoch 5/10
3/3 ██████████ 0s 13ms/step - accuracy: 0.3477 - loss: 1.1048 - val_accuracy: 0.2083 - val_loss: 1.1532
Epoch 6/10
3/3 ██████████ 0s 12ms/step - accuracy: 0.3971 - loss: 1.0959 - val_accuracy: 0.2917 - val_loss: 1.1481
1/1 ██████████ 0s 34ms/step
QCoreApplication::exec: The event loop is already running

```

Figure 10- Terminal Output

Throughout training, loss and accuracy values per epoch are clearly displayed in the terminal. Additionally, a GUI-based logging system is under development to track these outputs.

```

≡ ml_gui_20250525_220604.log
≡ ml_gui_20250525_220634.log
≡ ml_gui_20250525_220902.log
≡ ml_gui_20250525_221024.log
≡ ml_gui_20250525_221101.log
≡ ml_gui_20250525_221113.log
≡ ml_gui_20250525_221316.log
≡ ml_gui_20250525_221339.log
≡ ml_gui_20250525_221415.log
≡ ml_gui_20250525_221435.log
≡ ml_gui_20250525_221506.log
≡ ml_gui_20250525_221556.log
≡ ml_gui_20250525_221645.log
≡ ml_gui_20250525_221705.log
≡ ml_gui_20250525_221901.log
≡ ml_gui_20250525_222008.log
≡ ml_gui_20250525_222212.log

```

Figure 11-Real-Time Logging

All actions performed via the GUI, including errors and training steps, are logged in real-time into timestamped files. A future update will consolidate these logs into a single prioritized file.